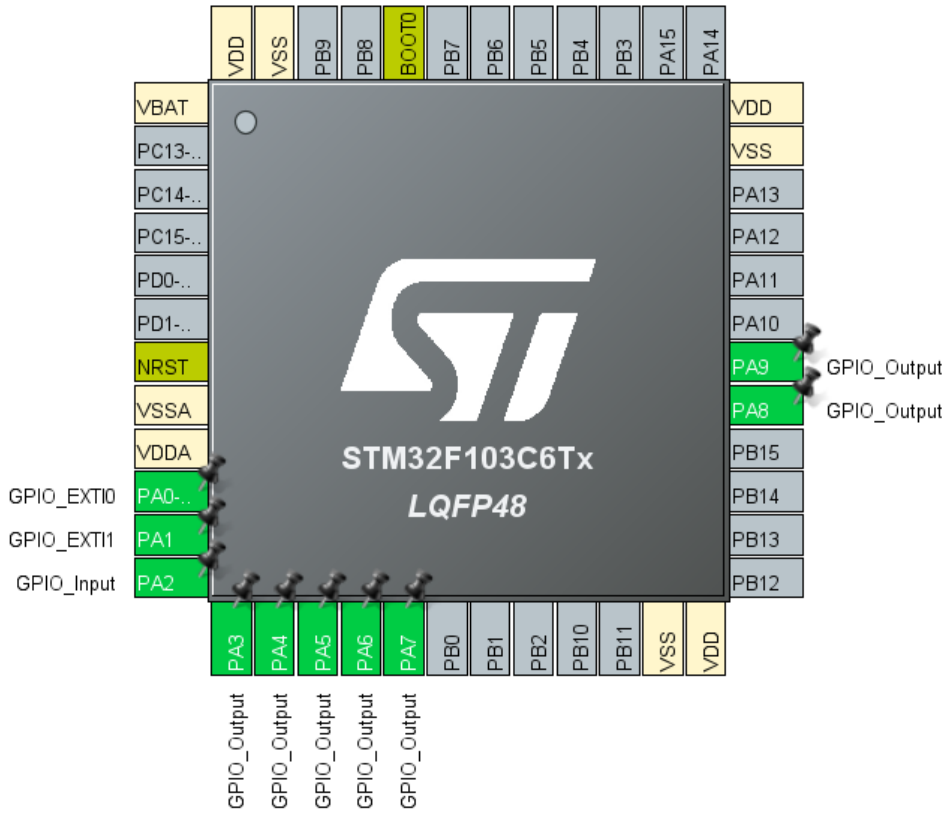
درس ابزار دقیق، دکتر نیری

تمرین سری سوم

غزل شریعت محرری-۸۱۰۱۰۲۴۶۸

سوال اول)

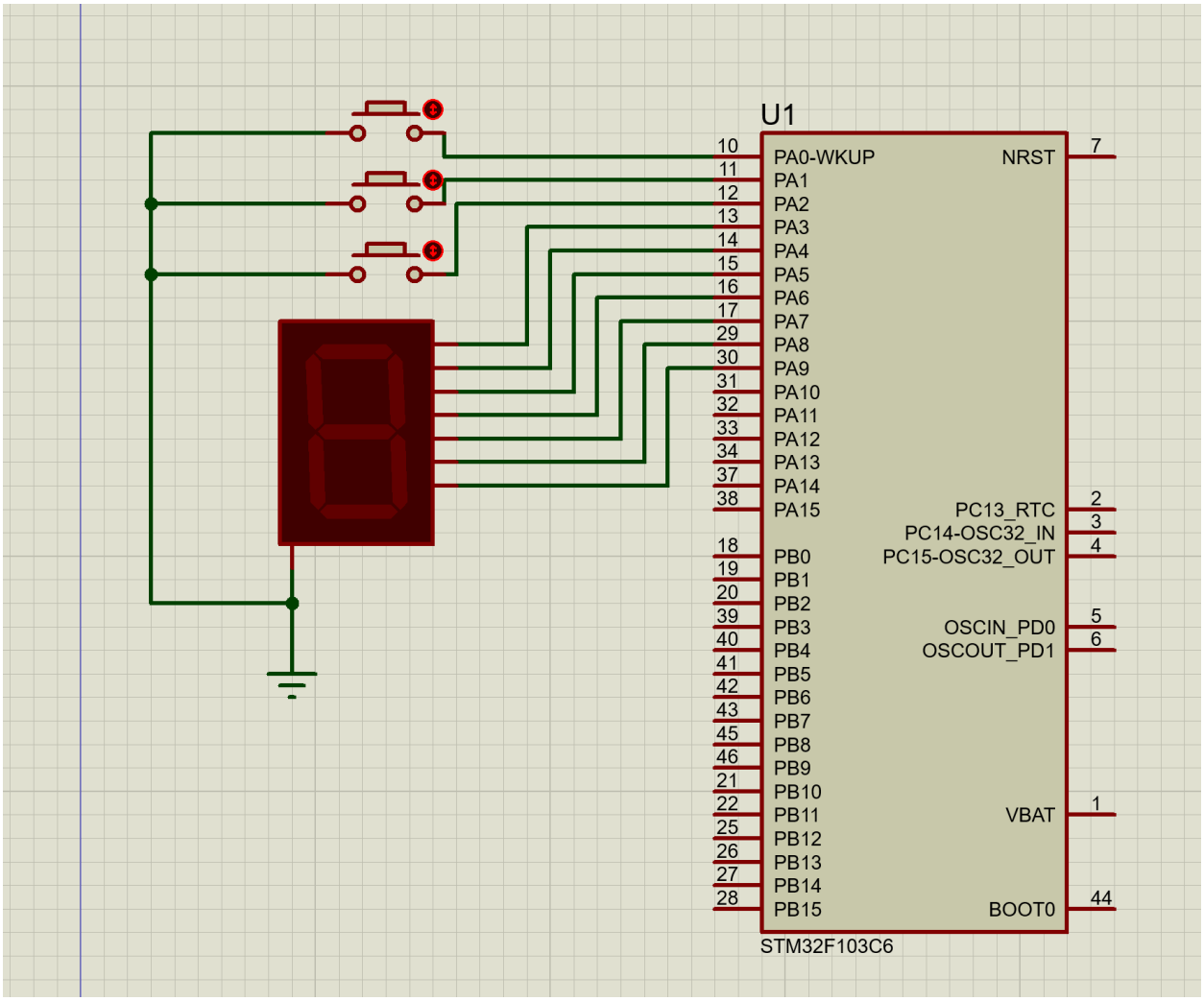
الف)



در این قسمت، پین‌های ورودی و خروجی را در میکروکنترلر مشخص میکنیم. پین های ورودی روی حالت pull-up قرار گرفته اند. (در حالت خنثی ولتاژشان صفر و بعد از فشرده شدن ولتاژ یک می‌شود.)

پین ۰ و ۱ و ۲ برای دکمه ها و پین ۳ تا ۹ برای سون سگمنت مشخص شده.

همینطور در پروتئوس ورودی و خروجی ها را به پین های مرتبط به آن ها در میکروکنترلر وصل کردیم.



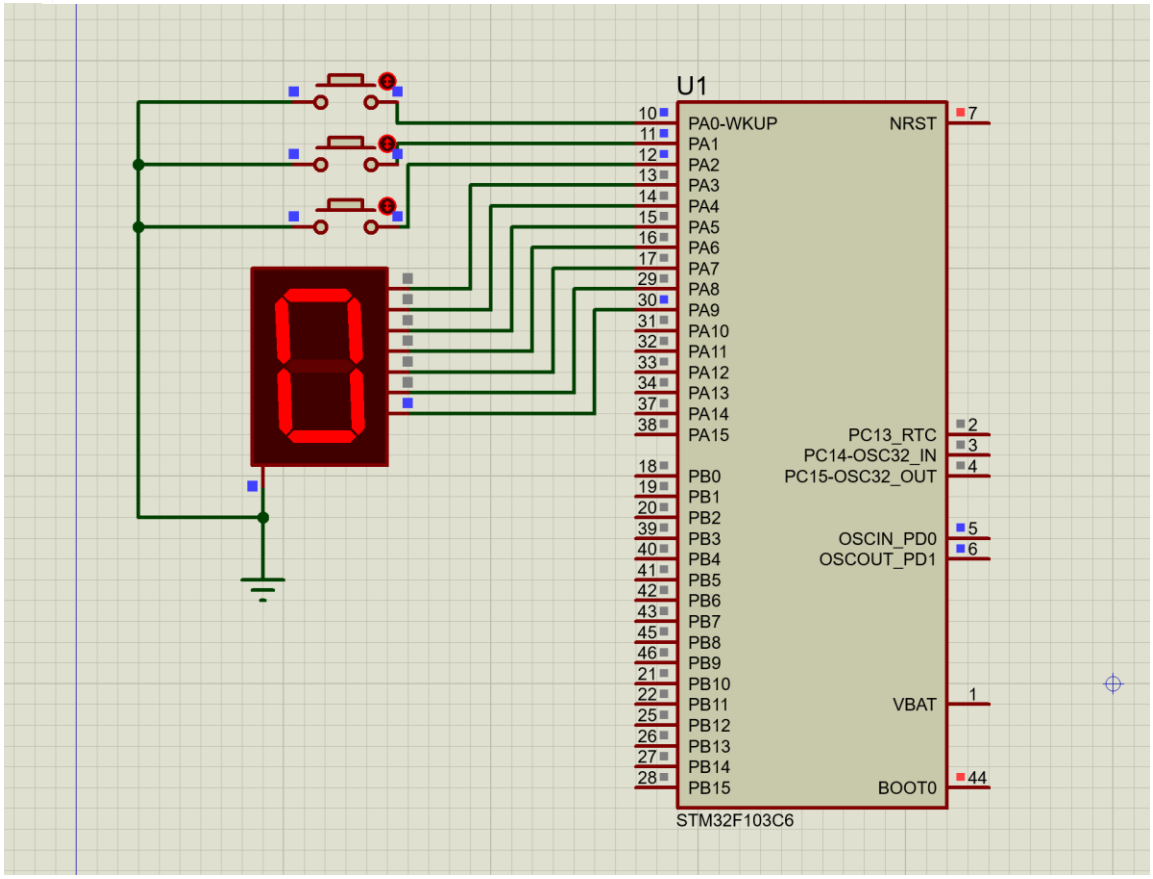
ب)

برای مشخص کردن حالت اولیه قبل از حلقه while در کد، مشخص میکنیم پین‌های a تا f در حالت روشن و پین g خاموش باشد. (عدد صفر)

```
  MX_GPIO_Init();
  /* USER CODE BEGIN 2 */
  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, GPIO_PIN_SET);   // Segment A ON
  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_SET);   // Segment B ON
  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);   // Segment C ON
  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET);   // Segment D ON
  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_SET);   // Segment E ON
  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET);   // Segment F ON
  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_RESET); // Segment G OFF
  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}
```



ج)

در این قسمت، از یک آرایه دو بعدی برای ذخیره کلید های روشن در هر عدد استفاده شده. (به دلیل راحت برقرار شدن اتصالات به میکروکنترلر سون سگمنت آینه شده است و کلید ها با این فرض روشن و خاموش شده اند.)

```
uint8_t numbers[10][7] = {
  {1,1,1,1,1,1,0}, // 0: Symmetric (No change)
  {0,0,0,0,1,1,0}, // 1: Lights up physical F and E (Top-Right and Bot-Right on mirror)
  {1,0,1,1,0,1,1}, // 2: Lights up A, F, G, C, D
  {1,0,0,1,1,1,1}, // 3: Lights up A, F, E, D, G
  {0,1,0,0,1,1,1}, // 4: Lights up B, F, E, G
  {1,1,0,1,1,0,1}, // 5: Lights up A, B, G, E, D
  {1,1,1,1,1,0,1}, // 6: Lights up A, B, C, D, E, G
  {1,0,0,0,1,1,0}, // 7: Lights up A, F, E
  {1,1,1,1,1,1,1}, // 8: Symmetric (No change)
  {1,1,0,1,1,1,1}  // 9: Lights up A, B, F, G, E, D
};
```

در ابتدای main دو فلگ تعریف شده، یکی برای مشخص کردن استیت برنامه (شمردن یا نشمردن) و دیگری برای carry out کانتر.

```
/* USER CODE BEGIN 1 */
  uint8_t is_counting = 0; // Flag to track if the button was pressed
  uint8_t counter = 0;
/* USER CODE END 1 */
```

```
while (1)
  {
      // 1. Read the button on PA2. If it is pressed, set the flag to 1.
      if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_2) == GPIO_PIN_RESET) {
          is_counting = 1;
      }

      // 2. If the flag is set, start the continuous counting process
      if (is_counting == 1) {
          // Write the array values to the 7 output pins
          HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, numbers[counter][0]); // Segment A
          HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, numbers[counter][1]); // Segment B
          HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, numbers[counter][2]); // Segment C
          HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, numbers[counter][3]); // Segment D
          HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, numbers[counter][4]); // Segment E
          HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, numbers[counter][5]); // Segment F
          HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, numbers[counter][6]); // Segment G

          // Increment the counter. If it goes past 9, reset it to 0.
          counter++;
          if (counter > 9) {
              counter = 0;
          }

          // Wait 1000 milliseconds (1 second) before looping again
          HAL_Delay(1000);
      }
  }
```

در حلقه while ابتدا صفر شدن دکمه را بررسی میکنیم، اگر فشرده شده بود وارد استیت شمردن می‌شویم. روشن یا خاموش بودن پین ها را از روی آرایه بررسی می‌کنیم و کانتر را هر بار جلو می‌بریم. یک دیلی یک ثانیه هم قرار دادیم تا اعداد قابل دیدن باشند.

د)

در این قسمت می‌خواهیم به سیستم interrupt اضافه کنیم. برای این کار ابتدا متغیرهای is_counting و counter را به صورت گلوبال تعریف می‌کنیم و متغیر جدید system_halted را تعریف می‌کنیم، تا مطمئن شویم زمانی که interrupt وارد شد دیگر به حالت شمارش قبل برنمیگردیم. تابع interrupt نیز به آن‌ها دسترسی داشته باشد. تابع HAL_GPIO_EXTI_Callback را تعریف میکنیم.

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_PIN)
{
        // Check if the interrupt was triggered by PA0
        if (GPIO_PIN == GPIO_PIN_0) {
            system_halted=1;
            is_counting = 0; // Stop the main loop from overriding the display

            // Display "1"
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, numbers[1][0]);
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, numbers[1][1]);
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, numbers[1][2]);
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, numbers[1][3]);
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, numbers[1][4]);
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, numbers[1][5]);
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, numbers[1][6]);
        }

        // Check if the interrupt was triggered by PA1
        else if (GPIO_PIN == GPIO_PIN_1) {
            system_halted=1;
            is_counting = 0; // Stop the main loop

            // Display "9"
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, numbers[9][0]);
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, numbers[9][1]);
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, numbers[9][2]);
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, numbers[9][3]);
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, numbers[9][4]);
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, numbers[9][5]);
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, numbers[9][6]);
        }
    }
```

تابع main را نیز به صورتی تغییر می‌دهیم تا فقط در زمانی دکمه ۲ را برای شروع شمارش چک کند که interrupt ندیده باشیم.

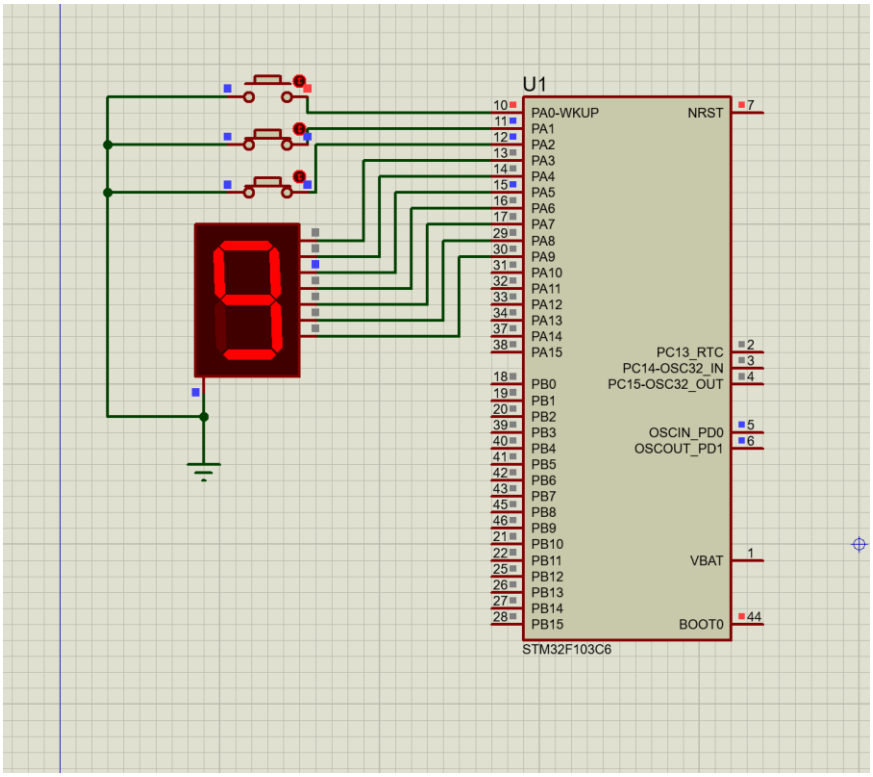
```
while (1)
  {
     // 1. Read the button on PA2. If it is pressed, set the flag to 1.
    if (system_halted==0){
      if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_2) == GPIO_PIN_RESET) {
          is_counting = 1;
      }}

      // 2. If the flag is set, start the continuous counting process
      if (is_counting == 1) {
          // Write the array values to the 7 output pins
          HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, numbers[counter][0]); // Segment A
          HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, numbers[counter][1]); // Segment B
          HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, numbers[counter][2]); // Segment C
          HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, numbers[counter][3]); // Segment D
          HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, numbers[counter][4]); // Segment E
          HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, numbers[counter][5]); // Segment F
          HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, numbers[counter][6]); // Segment G

          // Increment the counter. If it goes past 9, reset it to 0.
          counter++;
          if (counter > 9) {
              counter = 0;
          }

          // Wait 1000 milliseconds (1 second) before looping again
          HAL_Delay(1000);
      }
  }
```

در پروتئوس شبیه سازی می‌کنیم و مشاهده می‌کنیم به خوبی کار می‌کند.



سوال دوم)

الف)

واحد TIM مخفف Timer (زمان‌سنج) است. این واحد، یک سخت‌افزار داخلی کاملاً مستقل در میکروکنترلر است که وظیفه شمارش پالس‌های کلاک، ایجاد تاخیرهای دقیق، اندازه‌گیری زمان و تولید سیگنال‌های کنترلی مانند PWM را بر عهده دارد.

برای اضافه کردن آن به stm32 از تب timers یکی از timerها مثلا TIM1 را انتخاب کرده و یکی از چنل های آن را انتخاب می‌کنیم. کلاک مورد استفاده آن را internal clock قرار می‌دهیم. یعنی کلاک تایمر از کلاک اصلی میکروکنترلر استفاده می‌کند. کلاک مورد استفاده من طبق شماره دانشجویی 26MHz است. سیگنال PWM کلاک 1KHz باید داشته باشد این کلاک را از طریق پارامترهای PCS و ARR تولید می‌کنیم.

پارامتر PCS (Pre-scaler): مقسوم‌علیه فرکانس است. این رجیستر فرکانس اصلی و بسیار سریع میکروکنترلر را دریافت کرده و آن را تقسیم می‌کند یعنی با استفاده از یک counter فقط در carry-out کانتر خروجی پالس بدهد.

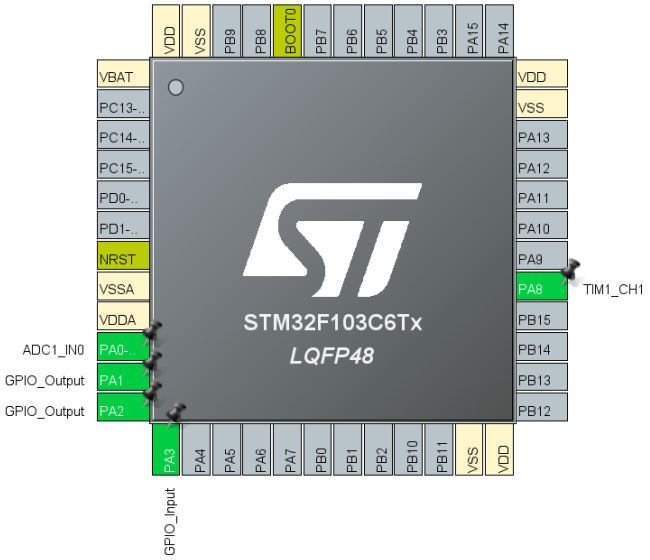
پارامتر ARR (Auto-Reload Register): سقف شمارش تایمر است. تایمر از صفر تا این عدد می‌شمارد و سپس دوباره به صفر برمی‌گردد (ریست می‌شود). زمانِ یک دور شمارش کامل تا رسیدن به این عدد، تعیین‌کننده فرکانس نهایی سیگنال PWM است.

$$Frequency_{PWM} = \frac{Frequency_{clock}}{(PSC\ +\ 1)\ (ARR\ +\ 1)}$$

$$1 = \frac{26000}{(PCS+1)(ARR+1)} \rightarrow PCS = 999, ARR = 25$$

ب)

برای خواندن مقدار پتانسیومتر به یک پایه ADC نیاز داریم. (PA0) و برای ورودی‌های درایور موتور به دو پایه خروجی از میکروکنترلر نیاز داریم. (PA1&PA2) تایمر PWM را نیز طبق قسمت قبل مشخص کردیم و طبق ستینگ میکروکنترلر خروجی آن از PA8 خارج می‌شود. برای ورودی دکمه فشاری ریورس نیز از PA3 استفاده می‌کنیم.



دو متغیر adc_value و motor_direction را به صورت گلوبال تعریف می‌کنیم.

در حلقه while ابتدا مقدار adc را می‌خوانیم و ۱۰ میلی ثانیه زمان برای خواندن آن به میکرو می‌دهیم.

```
HAL_ADC_Start(&hadc1);
HAL_ADC_PollForConversion(&hadc1, 10);
adc_value = HAL_ADC_GetValue(&hadc1);
```

سپس مقدار ۱۲ بیتیADC را به مقداری متناسب برای CCR تولید می‌کنیم. و مقدار PWM را با CCR جدید تولید می‌کنیم. سپس وضعیت دکمه فشاری را بررسی می‌کنیم و اگر فشرده شده بود جهت موتور رو برعکس می‌کنیم. برای برعکس کردن جهت چرخش IN0 و IN1 را برعکس می‌کنیم.

```
while (1)
{
  /* USER CODE END WHILE */
    // 1. Read the Potentiometer (ADC on PA0)
          HAL_ADC_Start(&hadc1);
          HAL_ADC_PollForConversion(&hadc1, 10);
          adc_value = HAL_ADC_GetValue(&hadc1);

          // 2. Map ADC value to your TIM1 ARR (which is currently set to 25)
          uint32_t pwm_duty = (adc_value * 25) / 4095;
          __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, pwm_duty); // Updates PA8 PWM

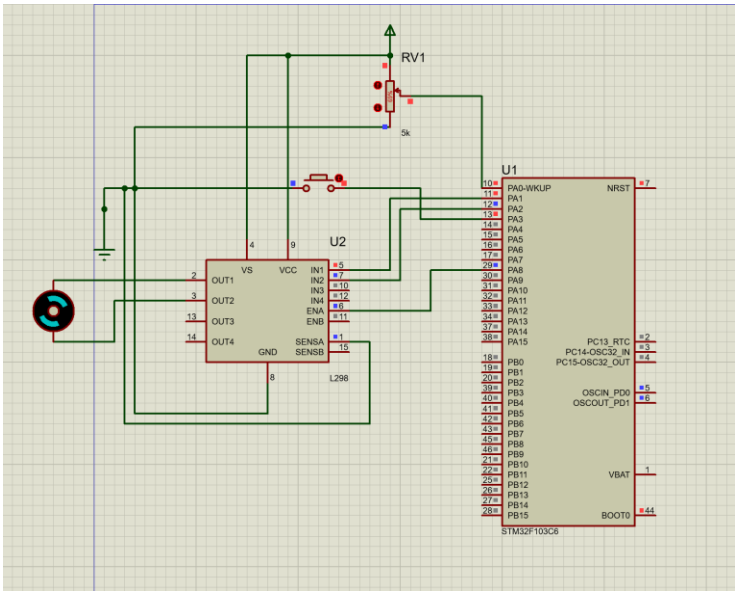
          // 3. Check the Reverse Button (PA3)
          // Because it is a Pull-Up, pressing it sends a RESET (0)
          if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_3) == GPIO_PIN_RESET) {
              HAL_Delay(200); // Simple button debounce so it doesn't flicker
              motor_direction = !motor_direction; // Flip the direction variable
          }

          // 4. Send Motor Direction to the L298 Driver (PA1 and PA2)
          if (motor_direction == 1) {
              HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, GPIO_PIN_SET);
              HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2, GPIO_PIN_RESET);
          } else {
              HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, GPIO_PIN_RESET);
              HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2, GPIO_PIN_SET);
          }

          HAL_Delay(50);
  /* USER CODE BEGIN 3 */
}
```

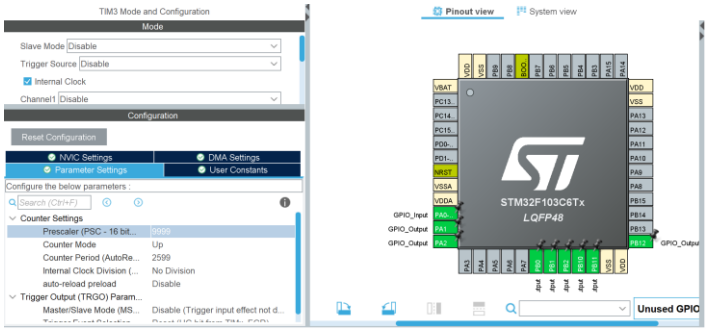
ج) پیاده سازی در پروتئوس

همانطور که انتظار می‌رفت در پروتئوس با فشردن دکمه جهت حرکت موتور تغییر، و با تغییر پتانسیومتر سرعت موتور تغییر می‌کند.



سوال سوم

کتابخانه LCD را include می‌کنیم و پایه ها را همانند صورت سوال مشخص می‌کنیم. همینطور دو متغیر گلوبال countdown و is_running را تعریف می‌کنیم.



برای ایجاد تایمر یک ثانیه، در تایمر ۳، ARR و PSC متناسب را انتخاب می‌کنیم تا کلاک ۱ هرتز تولید شود و enable interrupt را روشن می‌کنیم تا از این سیگنال بتوانیم به عنوان interrupt در کد استفاده کنیم.

همینطور مطابق صورت سوال، LCD را initialize می‌کنیم. تابع interrupt ای که برای شمارش یک ثانیه و تغییر خروجی متصل به LED و Buzzer تعریف کرده ایم را قبل از while به صدا می‌زنیم تا خودکار شروع به کار کند و شمارشش شروع شود.

تابع interrupt:

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM3) {
        if (is_running == 1) {
            if (countdown > 0) {
                countdown--;
                // Toggle LED and Buzzer every second
                HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_1);
                HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_2);
            }
            else if (countdown == 0) {
                // Time is up! Solid ON.
                HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, GPIO_PIN_SET);
                HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2, GPIO_PIN_SET);
            }
        }
    }
}
```

در حلقه while ابتدا ورودی logicstate را بررسی کرده و نوشته روی LCD را با توجه به state مشخص می‌کنیم.

```
while (1)
{
  /* USER CODE END WHILE */
    if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_SET) {
          is_running = 1;
      } else {
          // If LogicState is 0, reset everything!
          is_running = 0;
          countdown = 15;
          HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, GPIO_PIN_RESET); // LED OFF
          HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2, GPIO_PIN_RESET); // Buzzer OFF
      }

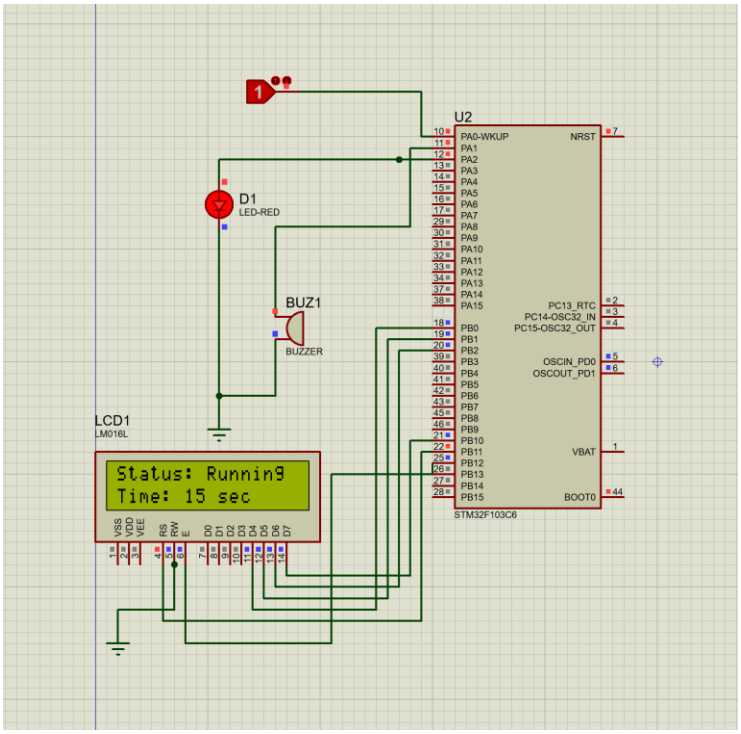
      char buffer[16];
      Lcd_cursor(&lcd, 0, 0);
      Lcd_string(&lcd, "Status:        ");
      Lcd_cursor(&lcd, 0, 8);

      if (is_running) Lcd_string(&lcd, "Running");
      else Lcd_string(&lcd, "Stopped");

      Lcd_cursor(&lcd, 1, 0);
      sprintf(buffer, "Time: %02d sec   ", countdown);
      Lcd_string(&lcd, buffer);

      HAL_Delay(100);
  /* USER CODE BEGIN 3 */
}
```

در پروتئوس نیز پس از شبیه سازی خروجی به درستی عملکرد سیستم شمارش معکوس را نشان می‌دهد.



سوال چهارم

الف)ADC

ADC یا Analog-to-Digital-Converter برای تبدیل سیگنال آنالوگ به دیجیتال استفاده می‌شود. در

میکروکنترلرها این پایه سیگنال آنالوگ را دریافت می‌کند و به یک سیگنال دیجیتال تبدیل می‌کند. برای اضافه

کردن آن به stm32 یکی از پایه ها را ADC input قرار می‌دهیم. مثلا پایه PA0 را ADC1-IN0 قرار می‌دهیم تا آن پایه خروجی پتانسیومتر را دریافت کند و به خروجی دیجیتال تبدیل کند.

در صورت سوالات و ویدیوها اشاره شده که ADC داخل میکروکنترلرهای مورد استفاده ما ۱۲ بیتی است و ولتاژ مرجع اینجا ۳.۳ ولت است پس رزولوشون:

$$Resolution = \frac{3.3}{4095} = 0.0008V = 8mV$$

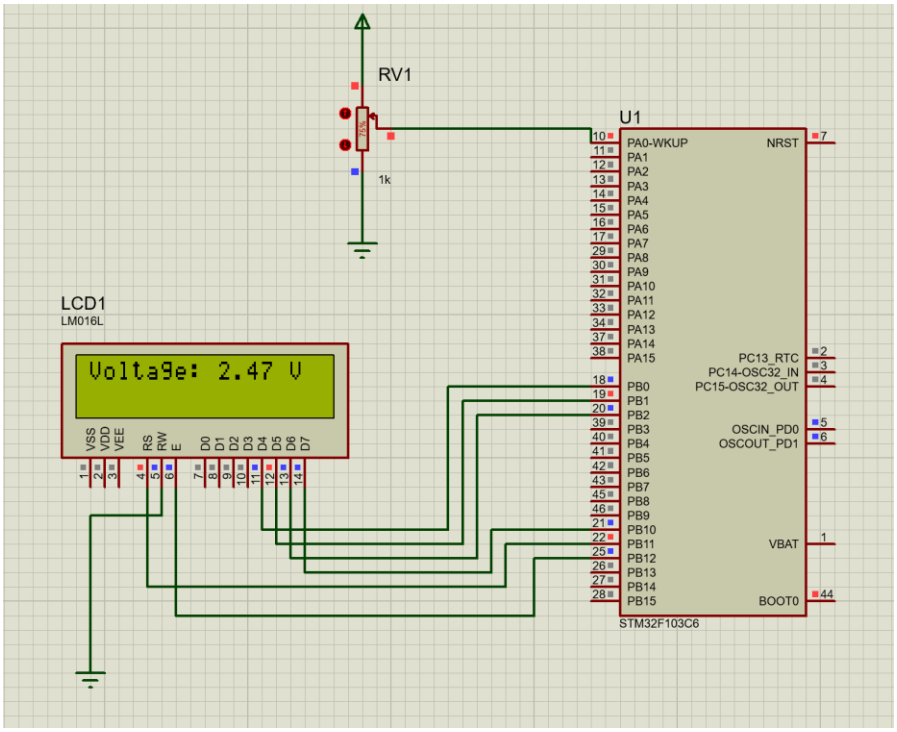
۸ میلی ولت می باشد.

ب)

```
while (1)
{
    HAL_ADC_Start(&hadc1);
    HAL_ADC_PollForConversion(&hadc1, 10);
    uint32_t raw_adc = HAL_ADC_GetValue(&hadc1);
    float voltage = (raw_adc * 3.3f) / 4095.0f;
    char line1[16];
    sprintf(line1, "Voltage: %.2f V   ", voltage);
    HAL_Delay(200);
    Lcd_cursor(&lcd, 0, 0);
    Lcd_string(&lcd, line1);
  /* USER CODE END WHILE */
  /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
}
```

همانند بخش قبل pin های LCD را مشخص کرده و آن را initialize می‌کنیم. داخل حلقه while ابتدا ADC را start می‌کنیم و ۱۰ میلی ثانیه به آن فرصت تبدیل می‌دهیم. پس از آن مقدار را دریافت می‌کنیم و از مقدار خام مقدار ولتاژ آن را بدست می‌آوریم. مقدار بدست آمده را در LCD می‌نویسیم.

شبیه سازی در پروتئوس:



ج)

```
while (1)
{
    HAL_ADC_Start(&hadc1);
        HAL_ADC_PollForConversion(&hadc1, 10);
        uint32_t raw_adc = HAL_ADC_GetValue(&hadc1);

        float voltage = (raw_adc * 3.3f) / 4095.0f;
        char line1[16];
        char line2[16];
        float physical_value = 0.0f;
        if (voltage >= 0.0f && voltage < 1.0f) {
            physical_value = 100.0f * voltage;
            sprintf(line1, "Sensor: LM35   ");
            sprintf(line2, "Temp: %.2f C   ", physical_value);
        }
        else if (voltage >= 1.0f && voltage < 2.0f) {
            physical_value = 33.3f * voltage;
            sprintf(line1, "Sensor: HIH4000");
            sprintf(line2, "Hum: %.2f %%   ", physical_value);
        }
        else if (voltage >= 2.0f && voltage < 3.0f) {
            physical_value = 100.0f * voltage;
            sprintf(line1, "Sensor: MPX4115");
            sprintf(line2, "Pres: %.2f hPa ", physical_value);
        }
        else if (voltage >= 3.0f && voltage <= 3.3f) {
            physical_value = 100.0f * (voltage / 3.3f);
            sprintf(line1, "Sensor: MQ-135 ");
            sprintf(line2, "AirQ: %.2f %%   ", physical_value);
        }
        Lcd_cursor(&lcd, 0, 0);
        Lcd_string(&lcd, line1);
        Lcd_cursor(&lcd, 1, 0);
        Lcd_string(&lcd, line2);

        HAL_Delay(200);
  /* USER CODE END WHILE */
  /* USER CODE BEGIN 3 */
}
```

به کد قسمت قبل، یک سری if statement اضافه می‌کنیم که با توجه به مقدار ولتاژ بدست آمده مشخص کنند سنسور از چه نوعی است و فرمت خروجی را درست کنند. همانند بخش قبل این داده‌ها را با LCD نمایش می‌دهیم.

د)

میکروکنترلر را حال به LCD متصل می‌کنیم و خروجی را مشاهده می‌کنیم. همینطور که مشاهده می‌شود به

درستی کار می‌کند و متناسب با مقدار پتانسیومتر و تشخیص نوع سنسور خروجی نمایش داده می‌شود.

